

武汉理工大学课程大作业

课程名称：	企业级应用软件设计与开发
项目名称：	智能穿搭助手 Agent
方向：	方向一： Agentic AI 原生开发
学号：	2025302917
姓名：	陈添乐
专业：	计算机技术
指导教师：	戚欣
提交日期：	2026 年 6 月 22 日

目录

- 一、 选题背景与设计思想.....1
 - 1.1 痛点分析.....1
 - 1.2 项目价值.....1
 - 1.3 技术路线.....2
- 二、 Specs 规格文档.....3
 - 2.1 Product Spec（产品规格）3
 - 2.1.1 用户角色.....3
 - 2.1.2 用户故事.....3
 - 2.1.3 功能列表.....5
 - 2.1.4 非功能需求.....6
 - 2.2 Architecture Spec（架构规格）7
 - 2.2.1 整体架构.....7
 - 2.2.2 模块职责.....7
 - 2.2.3 数据流设计.....8
 - 2.2.4 状态管理.....9
 - 2.2.5 可扩展性设计.....10
 - 2.3 API Spec（接口规格）10
 - 2.3.1 内部工具 API.....10
 - 2.3.2 外部 API 依赖13
 - 2.3.3 用户交互接口.....14
 - 2.4 Specs 文档小结16
- 三、 系统架构与设计.....16
 - 3.1 整体架构.....16

3.2 Agent 交互流程与数据流设计	17
四、关键实现与代码展示.....	17
4.1 Agent 核心循环	17
4.2 工具定义.....	19
4.2.1 天气工具.....	19
4.2.2 衣橱工具.....	20
4.3 配置文件.....	21
五、测试与评估.....	22
5.1 功能测试.....	22
5.1.1 天气查询.....	22
5.1.2 衣橱管理.....	22
5.2 Agent 行为评估	23
六、系统升级与扩展.....	23
七、课程总结.....	23

一、选题背景与设计思想

1.1 痛点分析

每天出门前思考“今天穿什么”是一个看似简单却长期未被很好解决的生活问题。这个决策过程虽然日常，却涉及多个维度的信息整合与判断，给用户带来不少隐性负担。

首先是信息割裂的问题。用户如果想做一个合理的穿搭决策，通常需要先打开天气类应用程序查看当天的温度、降水、风速和紫外线情况，再根据自己的记忆回想衣橱里有哪些可穿的衣物，同时还要结合当天的日程安排判断是否有重要会议、约会或者运动计划。这些信息分散在不同的渠道和大脑记忆的不同区域，用户需要在多个应用之间切换，同时还要依赖自己并不完全可靠的记忆。

其次是决策疲劳的问题。穿搭决策虽然单次看似不重要，但每天都要做，累积起来消耗了大量的认知资源。尤其是对于每天通勤的上班族或者课业繁重的学生来说，早上时间本就紧张，还要在有限的几分钟内做出一个既舒适又得体的穿搭决定，这本身就是一种不必要的负担。

再次是缺乏个性化的问题。现有的穿搭建议来源，无论是时尚博主的推荐还是网站的搭配指南，都是面向大众的通用方案。这些建议不会知道用户的衣橱里具体有什么衣服，不会知道用户不喜欢穿某一种颜色或者某一种材质，也不会知道用户的体型特点和风格偏好。因此这些建议往往看起来很好，实际却不适用于具体的个人。

最后是经验难以沉淀的问题。每次穿搭决策都是独立的，用户今天觉得不错的搭配，可能过几天就忘记了。好的穿搭经验无法被系统记录和复用，导致用户每次都要从头开始思考，无法在过往经验的基础上不断优化。

1.2 项目价值

针对上述痛点，本项目设计并实现一个智能穿搭助手智能体，其价值体现在以下几个层面。

对个人用户而言，首先是效率的提升。用户只需要用自然语言说一句“今天穿什么”，智能体就能自动完成天气查询、衣橱检索、场合适配和搭配推理等一系列工作，将穿搭决策的时间从几分钟压缩到几秒钟。

其次是决策质量的提升。智能体基于实时天气数据、用户真实衣橱信息和明确的搭配规则给出建议，比用户凭感觉做出的选择更加可靠和一致。

第三是学习能力。智能体可以记住用户不喜欢穿什么、喜欢什么风格，随着使用次数的增加，推荐结果会越来越贴合用户的真实偏好，真正做到越用越懂你。

第四是趣味性，智能体可以给穿搭过程增加一些探索和惊喜感，帮助用户发现自己衣橱中原本被忽略的搭配可能。

1.3 技术路线

本项目的技术路线围绕智能体核心能力展开，整体架构采用主从式智能体架构。

在智能体框架选择上，本项目使用 **LangGraph** 作为核心开发框架。**LangGraph** 提供了基于图结构的状态管理能力，可以清晰定义智能体处理用户请求的每一个步骤，包括意图识别、数据获取、推理决策和结果生成，同时天然支持多轮对话中的状态保持和流程分支控制。

在工具调用方面，智能体需要具备调用外部能力的能力。具体来说，智能体将定义两个核心工具：天气查询工具用于调用天气应用程序编程接口获取实时天气数据并做初步解读，衣橱检索工具用于从用户衣橱中查找符合当前天气和场合条件的衣物。

在记忆机制方面，本项目采用分层记忆设计。对话窗口记忆用于维持当前会话的上下文，让智能体能够理解用户在当前对话中的连续提问。衣橱数据以 **JSON** 格式本地存储，检索采用规则过滤（温度范围匹配 + 场合标签匹配）。

在检索增强生成方面，衣橱检索模块采用规则过滤技术。当用户询问穿搭建议时，智能体会将当前的天气条件和场合类型组合成过滤条件，在衣橱中筛选符合条件的衣物，然后将检索结果作为上下文传递给大语言模型，让模型基于真实存在的衣物生成建议，而不是凭空编造。

在智能体架构设计上，本项目采用主从式结构，以 **LangGraph** 作为编排框架。核心只有一个主控智能体，负责整个对话流程的编排和执行；天气工具和衣橱工具作为“从属”组件，由主控智能体按需调用。主控智能体首先对用户输入进行意图分类，识别出用户是想问穿搭、查天气还是管理衣橱。根据意图类型，主控智能体依次调用相应的工具函数完成子任务：当需要穿搭建议时，先调用天气工具获取实时天气数据，再调用衣橱工具检索符合条件的衣物，最后将天气数据和衣橱候选整合到提示词中，调用大语言模型生成最终的穿搭建议。这种设计的优势在于架构简洁，所有状态由主控智能体统一管理，避免了多智能体之间复杂的通信和协调开销。工具函数作为智能体的“从属组件”独立封装，便于单独测试和替换。

在开发流程上，本项目遵循规格驱动开发方法论，先完成产品规格、架构规格和接口规格的设计文档，再进行代码实现。在交付节奏上，第一周完成最小可行产品版本，实现天气查询加固定建议的基础功能，第二周完成增强版本，加入衣橱检索和个性化推荐，最终完成完整的系统演示和文档撰写。

在部署和评估方面，项目支持本地命令行运行和简单的网页界面两种使用方式。评估将包括功能测试验证各个工具是否正确工作，效果评估对比智能体建议与人工建议的质量差异，以及性能测试记录每次请求的耗时和令牌消耗。可观测性方面将通过日志记录智能体的每一步决策过程，便于调试和改进。

二、Specs 规格文档

2.1 Product Spec（产品规格）

2.1.1 用户角色

本项目定义以下用户角色：

普通用户：已完成衣橱初始化的用户，可以使用完整的穿搭推荐、衣橱管理和偏好学习功能。

访客：尚未初始化衣橱的用户，仅可使用基于天气的基础穿搭建议，无法获得个性化推荐。

2.1.2 用户故事

以下是本系统支持的用户故事，按功能模块组织。

穿搭推荐模块

作为一个普通用户，我希望用自然语言提问“今天穿什么”，系统能够自动结合天气和我的衣橱给出建议，以便我不需要手动查询多个信息源。

作为一个普通用户，我希望系统能根据当天的日程场合（如会议、约会、运动）调整推荐结果，以便我在不同场景下都能穿着得体。

作为一个普通用户，我希望系统在给出建议的同时说明推荐理由，例如“因为今天降温且有雨，推荐你穿防水外套”，以便我理解并信任系统的判断。

衣橱管理模块

作为一个普通用户，我希望能够查看当前衣橱中的所有衣物，以便了解自己拥有什么。

作为一个普通用户，我希望能够添加新衣物到衣橱，包括填写衣物的名称、类别、颜色、材质和适宜温度范围，以便衣橱数据保持完整。

作为一个普通用户，我希望能够删除不再拥有的衣物，以便衣橱数据保持准确。

作为一个普通用户，我希望能够更新衣物的信息，例如修改衣物的颜色或材质，以便纠正录入错误。

偏好学习模块

作为一个普通用户，我希望系统能记住我不喜欢的颜色和材质，例如我不喜欢穿红色或者不喜欢羊毛材质，以便后续推荐自动避开这些衣物。

作为一个普通用户，我希望系统能记住我偏好的风格，例如我更喜欢休闲风格而不是正式风格，以便推荐结果更贴合我的审美。

作为一个普通用户，我希望能够查看和手动删除已记录的偏好，以便纠正系统错误记忆的偏好。

基础功能模块

作为一个访客，我希望在不注册和初始化衣橱的情况下也能获得基于天气的基础穿搭建议，以便快速体验系统功能。

作为一个用户，我希望能够获得当日天气的简要信息，包括温度、天气状况、湿度和风速，以便了解室外环境。

2.1.3 功能列表

本节采用 MoSCoW 方法对功能进行优先级划分。

Must Have（必须实现，MVP 必备）

天气查询功能：系统能够调用外部天气应用程序编程接口，获取指定城市的实时温度、天气状况、湿度、风速和紫外线指数。

基于天气的基础穿搭建议：系统能够根据温度区间给出通用的穿衣建议，例如温度低于十度建议穿羽绒服，温度在十度到二十度之间建议穿薄外套。

自然语言对话交互：用户可以通过命令行或简单的网页界面输入自然语言问题，系统能够理解并响应。

环境变量配置：所有应用程序编程接口密钥必须通过环境变量配置，不得硬编码在代码中。

Should Have（应该实现，增强版包含）

衣橱管理功能：用户能够查看、添加、删除和更新衣橱中的衣物，数据存储在本地的 JSON 文件或轻量级数据库中。

基于衣橱的个性化推荐：系统能够从用户衣橱中检索符合条件的衣物，并基于真实存在的衣物生成推荐，而非凭空编造。

场合适配功能：系统能够识别用户提问中隐含的场合信息，例如“面试”“约会”“运动”，并根据场合调整推荐策略。

多轮对话上下文：系统能够记住当前对话中用户之前的提问和回答，支持连续追问，例如用户问“那明天呢”时自动基于明天的天气推荐。

Could Have（可以实现，时间充裕时加入）

偏好记忆功能：系统能够记录用户明确表达的不喜欢信息，并在后续推荐中自动排除相关衣物。

历史搭配记录：系统能够保存用户确认过的搭配方案，并支持查看和复用历史搭配。

推荐结果反馈：用户可以对推荐结果进行点赞或点踩，系统据此调整后续推荐的权重。

Won't Have（本次不做）

社交分享功能：用户可以将搭配分享到社交媒体。

多用户系统：本次仅支持单用户，用户身份通过固定的配置文件识别。

图像识别录入：用户拍照自动识别衣物信息并录入衣橱。

移动端应用程序：本次仅支持命令行和网页界面。

2.1.4 非功能需求

性能需求

单次请求的端到端响应时间应控制在简单查询 5 秒内，复杂查询 10 秒内，其中单次大语言模型推理部分不超过三秒。衣橱检索的响应时间应在五百毫秒以内。

可用性需求

系统应提供清晰的错误提示，当天气应用程序编程接口调用失败或大语言模型服务不可用时，应给出友好的错误信息而非直接崩溃。

安全需求

所有应用程序编程接口密钥必须通过环境变量配置，严禁硬编码。系统不收集用户的敏感个人信息，衣橱数据仅存储在本地。

可扩展性需求

衣橱存储模块应设计为可替换的接口，初期使用 JSON 文件，后续可无缝迁移到向量数据库或关系型数据库。工具函数的定义应遵循统一规范，便于新增或修改工具。

可观测性需求

系统应记录每次请求的关键步骤耗时，包括天气接口调用耗时、衣橱检索耗时和大语言模型调用耗时，便于性能分析和问题定位。

2.2 Architecture Spec（架构规格）

2.2.1 整体架构

本系统采用**主从式智能体架构**，以 **LangGraph** 作为核心编排框架。整体上分为五个层次：用户交互层、智能体编排层、智能体执行层、工具服务层和数据存储层。

用户交互层负责接收用户的自然语言输入并展示系统的回复结果。**MVP 阶段**采用命令行界面，增强版本升级为基于 **Streamlit** 的简单网页界面。

智能体编排层是系统的核心调度中枢，基于 **LangGraph** 构建。该层维护整个对话的状态，根据用户输入识别意图，决定调用哪个智能体或工具，并协调数据流转。

智能体执行层包含两个核心智能体：

- **主控智能体**：负责意图分类、路由调度、状态管理和工具调用。
- **时尚顾问智能体**：负责综合天气数据、衣橱候选和用户偏好，生成最终的穿搭建议。

工具服务层封装了智能体可调用的具体工具函数：

- **天气工具**：封装对和风天气 API 的调用，返回结构化天气数据。
- **衣橱工具**：封装对衣橱数据的增删改查和规则过滤检索。

数据存储层负责持久化数据：

- 衣橱数据以 **JSON 文件**或 **SQLite 数据库**存储。
- 对话状态由 **LangGraph** 在内存中维护（支持多轮对话）。

2.2.2 模块职责

主控 Agent (Main Controller)

主控智能体负责整个对话流程的编排。它接收用户的原始输入，通过大语言模型进行意图分类，识别出用户是想问穿搭、查天气、管理衣橱还是记录偏好。根据意图类型，主控智能体执行以下动作：

- **穿搭建议意图**：调用天气工具获取天气数据 → 调用衣橱工具检索候选衣物 → 调用时尚顾问智能体生成建议。
- **天气查询意图**：直接调用天气工具，将结果格式化后返回。
- **衣橱管理意图**：直接调用衣橱工具（增删改查），将操作结果返回。

主控智能体还负责维护 LangGraph 中的状态对象，确保数据在节点之间正确传递，并管理多轮对话的上下文历史。

时尚顾问 Agent (Fashion Advisor Agent)

时尚顾问智能体负责最终的推荐生成。它接收主控智能体传递的天气数据、衣橱候选列表和场合信息，结合用户偏好，调用大语言模型生成穿搭建议。时尚顾问智能体还需要进行冲突检测，例如检查推荐的衣物组合是否存在明显的搭配问题，或者推荐的衣物是否与用户明确表达的不喜欢冲突。

天气工具 (Weather Tool)

天气工具专门负责与天气相关的任务。它接收城市名称作为输入，调用和风天气 API 获取实时天气数据，并对数据进行初步解读。例如当温度为二十二度时，标注为"舒适温度"；当紫外线指数大于五时，标注为"需要注意防晒"。天气工具的输出是结构化的天气信息加上解读标签，供时尚顾问智能体使用。

衣橱工具 (Wardrobe Tool)

衣橱工具负责衣橱的所有操作。它支持四种类型的任务：查询衣物列表、添加新衣物、删除衣物、更新衣物信息。其中最核心的是检索任务：衣橱工具接收天气条件、场合类型和用户偏好作为查询条件，基于规则过滤（温度范围匹配 + 场合标签匹配 + 偏好排除）返回符合条件的衣物列表。

2.2.3 数据流设计

以用户提问"今天下午有个面试，帮我看看穿什么"为例，数据流如下：

第一步，用户输入进入主控智能体，主控智能体将输入传递给意图分类器，识别出意图为"穿搭建议"且隐含场合信息"面试"。

第二步，主控智能体调用天气工具，传入默认城市或用户之前设置的城市。天气工具调用和风天气 API，获取实时天气数据，返回温度二十五度、晴天、湿度百分之四十、微风，以及解读标签"舒适温度，注意防晒"。

第三步，主控智能体调用衣橱工具，传入天气数据（温度二十五度、晴天）和场合信息（面试）。衣橱工具构建查询条件：适宜温度覆盖二十五度，场合标签包含“正式”或“面试”，执行规则过滤，返回候选衣物列表：白色衬衫、深蓝色西裤、黑色皮鞋。

第四步，主控智能体调用时尚顾问智能体，传入天气数据、候选衣物列表和面试场合信息。时尚顾问智能体调用大语言模型，综合所有信息生成建议："今天二十五度晴天，建议穿白色衬衫搭配深蓝色西裤和黑色皮鞋，既正式又不会太热。"同时给出推荐理由。

第五步，时尚顾问智能体的输出返回给主控智能体，主控智能体将其作为最终回复返回给用户。

第六步，对话结束后，主控智能体将本次交互记录到对话历史中，用于支持多轮对话上下文。

2.2.4 状态管理

LangGraph 中维护的状态对象包含以下字段：

用户原始输入字段保存用户本次提问的原始文本。

意图字段保存意图分类器的识别结果，可能的值包括穿搭建议、天气查询、衣橱管理、偏好设置。

天气数据字段保存天气智能体返回的结构化天气信息，包含温度、天气状况、湿度、风速、紫外线指数和解读标签。

衣橱候选项字段保存衣橱智能体检索返回的衣物列表，每个衣物包含名称、类别、颜色、材质、适宜温度范围等属性。

最终输出字段保存时尚顾问智能体生成的最终回复文本。

对话历史字段保存当前会话中用户和系统的所有交互记录，用于支持多轮对话中的上下文理解。

错误信息字段保存执行过程中出现的错误信息，用于异常处理和用户提示。

字段	说明
user_input	用户原始输入

intent	意图类型（outfit/weather/wardrobe）
weather_data	天气数据
occasion	提取的场合信息
wardrobe_candidates	衣橱候选列表
final_output	最终回复
chat_history	对话历史

2.2.5 可扩展性设计

衣橱存储接口采用抽象类设计，定义了增删改查的标准方法。当前实现使用 JSON 文件存储，未来可以无缝替换为 SQLite 或 PostgreSQL 而无需修改上层代码。

衣橱检索接口采用规则过滤设计，当前基于温度范围匹配、场合标签匹配和偏好排除进行候选衣物的筛选。这种设计简单直接，满足 MVP 阶段的需求。未来可扩展为向量检索，支持基于语义的更智能匹配，例如用户输入“找一件适合面试的深色上衣”时，能够通过语义理解返回正确结果。

工具函数通过装饰器注册到工具注册表，新增工具只需编写函数并添加装饰器，无需修改主控智能体的代码。

智能体采用配置化设计，智能体的名称、使用的工具列表、系统提示词都通过配置文件定义，便于调整和实验不同的智能体组合。

2.3 API Spec（接口规格）

2.3.1 内部工具 API

所有工具函数以 Python 函数的形式实现，通过 LangChain 的 @tool 装饰器注册。

工具一：获取天气

- 工具名称：get_weather

- **功能描述:** 获取指定城市的实时天气数据, 包括温度、天气状况、湿度、风速和紫外线指数, 并返回适合穿搭的解读提示。
- **输入参数:**
 - **city (str, 必填):** 城市名称, 例如"北京""上海"。系统内部通过城市名称查询表或 GeoAPI 转换为和风天气城市代码。
- **输出格式:** 返回 dict, 包含:
 - **temp (int):** 摄氏温度
 - **condition (str):** 天气状况, 如"晴""雨""多云"
 - **humidity (int):** 相对湿度百分比
 - **wind_speed (str):** 风速等级, 如"微风""强风"
 - **uvi (int):** 紫外线指数
 - **tips (str):** 穿搭解读提示, 如"舒适温度, 注意防晒"

工具二: 检索衣橱

- **工具名称:** search_wardrobe
- **功能描述:** 根据当前的天气条件、场合类型和用户偏好, 从用户衣橱中检索最匹配的衣物列表。
- **输入参数:**
 - **temp (int, 必填):** 当前温度, 用于匹配衣物的适宜温度范围。
 - **condition (str, 必填):** 天气状况, 如"晴""雨", 用于判断是否需要防雨、防晒等。
 - **occasion (str, 选填):** 场合类型, 可选值为 casual (休闲)、work (工作)、interview (面试)、date (约会)、sports (运动)、formal (正式), 默认为 casual。用户输入中的自由文本 (如"有个会"、"见客户") 由主控智能体的意图分类器映射到标准枚举值。
 - **limit (int, 选填):** 返回的最大衣物数量, 默认为 8 件, 确保有足够的候选供搭配组合。
- **输出格式:** 返回 list, 每个元素为 dict, 包含:

- id (str): 衣物唯一标识 (UUID)
- name (str): 衣物名称
- category (str): 类别, 如"上衣""裤子""外套""鞋"
- color (str): 颜色
- material (str): 材质
- suitable_temp_min (int): 适宜最低温度
- suitable_temp_max (int): 适宜最高温度
- occasion_tags (list): 场合标签列表, 如 ["work", "formal"]

工具三: 添加衣物

- 工具名称: add_clothing
- 功能描述: 向用户衣橱中添加一件新衣物。
- 输入参数:
 - name (str, 必填): 衣物名称
 - category (str, 必填): 类别
 - color (str, 必填): 颜色
 - material (str, 必填): 材质
 - suitable_temp_min (int, 选填): 适宜最低温度, 默认为 0
 - suitable_temp_max (int, 选填): 适宜最高温度, 默认为 40
 - occasion_tags (list, 选填): 场合标签列表, 默认为 ["casual"]
- 输出格式: 返回 dict, 包含 success (bool)、id (str, UUID 生成)、error (str)

工具四: 删除衣物

- 工具名称: delete_clothing
- 功能描述: 从用户衣橱中删除一件衣物。
- 输入参数:

- `clothing_id` (str, 必填): 要删除的衣物唯一标识
- **输出格式:** 返回 dict, 包含 `success` (bool)、`error` (str)

工具五: 查询所有衣物

- **工具名称:** `list_wardrobe`
- **功能描述:** 返回用户衣橱中的所有衣物列表, 不进行任何过滤。
- **输入参数:** 无
- **输出格式:** 返回 list, 元素格式与检索衣橱工具相同。

工具六: 更新衣物

- **工具名称:** `update_clothing`
- **功能描述:** 更新用户衣橱中已有衣物的信息。
- **输入参数:**
 - `clothing_id` (str, 必填): 要更新的衣物唯一标识
 - `name` (str, 选填): 新名称, 不填则保持原值
 - `category` (str, 选填): 新类别
 - `color` (str, 选填): 新颜色
 - `material` (str, 选填): 新材质
 - `suitable_temp_min` (int, 选填): 新的适宜最低温度
 - `suitable_temp_max` (int, 选填): 新的适宜最高温度
 - `occasion_tags` (list, 选填): 新的场合标签列表
- **输出格式:** 返回 dict, 包含 `success` (bool)、`error` (str)

2.3.2 外部 API 依赖

天气 API (和风天气)

- **接口说明:** 使用和风天气开发版 API 或免费版 API。

- **城市代码转换：**和风天气使用数字城市代码而非城市名称。系统通过以下方式处理：
 - 方式一：内置常用城市名称到城市代码的映射表（覆盖中国主要城市）。
 - 方式二：对于映射表未覆盖的城市，调用和风天气 **GeoAPI** 将城市名称转换为城市代码。
- **请求方式：**HTTP GET，携带 **API Key** 作为查询参数。
- **返回数据：**JSON 格式，包含温度、天气状况、湿度、风速等字段。系统提取所需字段并转换为内部统一的天气数据格式。
- **限流处理：**免费版有 QPM 限制，系统应实现简单的请求缓存（如缓存 10 分钟内的同一城市天气数据）以减少 API 调用次数。

大语言模型 API（DeepSeek）

- **接口说明：**使用 DeepSeek API，调用方式为 OpenAI 兼容的对话补全接口。
- **模型选择：**
 - 意图分类：使用 **deepseek-chat (DeepSeek-V3)**，温度参数 0.3，确保分类稳定。
 - 穿搭建议生成：使用 **deepseek-chat (DeepSeek-V3)**，温度参数 0.5，在保证合理性的基础上提供一定多样性。
- **请求体：**包含 **model**、**messages**、**temperature** 字段。系统将工具函数定义和调用结果封装在 **messages** 中，由大语言模型决定调用工具或生成回复。
- **Token 管理：**系统应记录每次 LLM 调用的输入/输出 Token 数，用于成本估算和性能分析。

2.3.3 用户交互接口

MVP 阶段（命令行）

- 通过命令行界面直接交互，无需 HTTP 接口。

- 用户输入自然语言，系统直接输出回复。
- 启动命令：python main.py 或 python cli.py

增强版阶段（Streamlit Web + HTTP API）

对话接口

- 端点路径：/chat
- 请求方法：POST
- 请求头：Content-Type: application/json
- 请求体：

JSON

```
{
  "message": "用户输入文本",
  "user_id": "用户标识（可选，默认 default）",
  "session_id": "会话标识（可选，默认新生成）"
}
```

- 响应体：

JSON

```
{
  "reply": "系统回复文本",
  "suggestions": ["推荐衣物名称列表（可选）"],
  "reasoning": "推荐的推理过程（可选）",
  "elapsed_ms": 1234
}
```

健康检查接口

- 端点路径：/health

- 请求方法: GET
- 响应体: {"status": "ok"}

2.4 Specs 文档小结

本章完成了智能穿搭助手 Agent 的完整规格定义。产品规格部分定义了用户角色、用户故事、功能优先级和非功能需求，明确了系统要做什么以及做到什么程度。架构规格部分定义了整体架构、模块职责、数据流和状态管理，明确了系统如何组织以及各部分如何协作。接口规格部分定义了内部工具接口、外部依赖接口和用户交互接口，明确了系统与外界交互的具体协议。以上规格文档将作为后续代码实现的唯一依据，体现规格驱动开发的核心方法论。

三、系统架构与设计

3.1 整体架构

本系统采用基于 LangGraph 的主从智能体架构，分为用户交互层、智能体编排层、工具服务层和数据存储层。

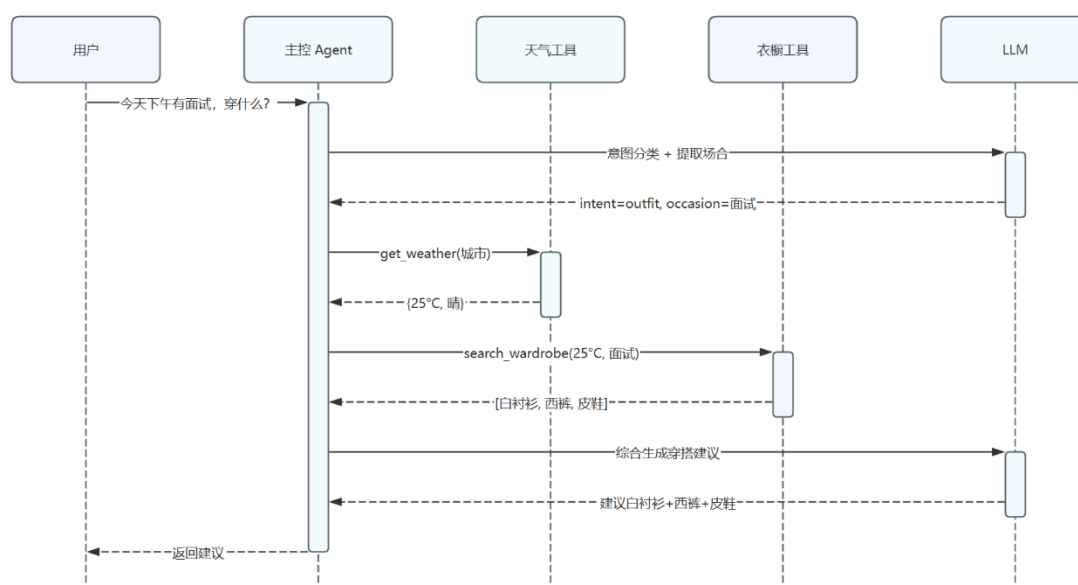


核心组件说明：

- **主控 Agent:** 基于 LangGraph StateGraph 构建，负责意图分类、工具调度和状态管理。
- **天气工具:** 封装和风天气 API 调用，返回温度、天气状况及穿搭提示。
- **衣柜工具:** 管理衣柜 JSON 文件的增删改查，支持基于温度和场合的规则过滤检索。

3.2 Agent 交互流程与数据流设计

以“今天下午有面试，穿什么？”为例：



四、关键实现与代码展示

Agent 核心循环、工具定义、配置文件、AI IDE 使用截图

4.1 Agent 核心循环

系统采用 **LangGraph StateGraph** 实现 Agent 核心循环，包含以下关键点：

```
# 核心节点定义
```

```

self.graph.add_node("entry_node", self.entry_node) # 入口节点

self.graph.add_node("intent_classifier", self.intent_classifier) # 意图分类

self.graph.add_node("weather_tool_node", self.weather_tool_node) # 天气工具

self.graph.add_node("wardrobe_search_node", self.wardrobe_search_node) # 衣橱检索

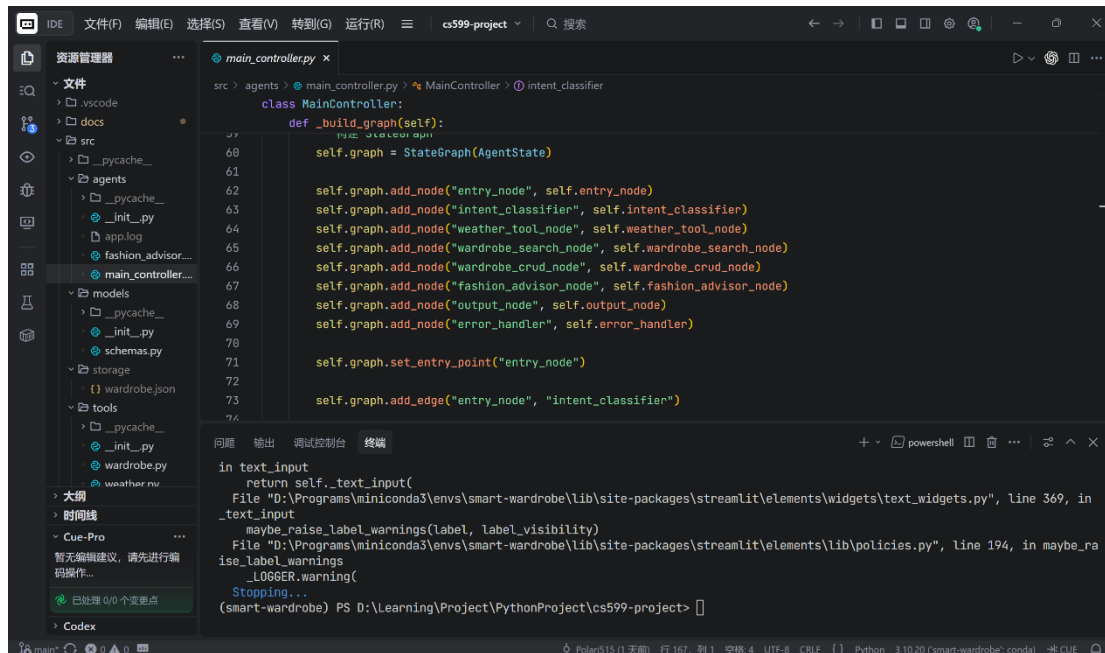
self.graph.add_node("wardrobe_crud_node", self.wardrobe_crud_node) # 衣橱 CRUD

self.graph.add_node("fashion_advisor_node", self.fashion_advisor_node) # 时尚顾问

self.graph.add_node("output_node", self.output_node) # 输出组装

self.graph.add_node("error_handler", self.error_handler) # 错误处理

```



意图分类核心逻辑（规则 + LLM 双层判断）：

```

# src/agents/main_controller.py

def _classify_intent_by_rules(self, user_input: str) -> Optional[Dict]:

    """规则优先的意图分类，减少 LLM 依赖"""

    weather_keywords = ["天气", "温度", "气温", "冷", "热", "晴", "雨", "雪"]

```

```

wardrobe_keywords = ["添加", "删除", "查看", "衣橱", "衣柜", "修改", "更新", "列表"]

outfit_keywords = ["穿什么", "搭配", "建议穿", "怎么穿", "穿搭"]

# 优先级: 天气查询 > 衣橱管理 > 穿搭建议

if any(keyword in normalized for keyword in weather_keywords):

    return {"intent": "weather", ...}

elif any(keyword in normalized for keyword in wardrobe_keywords):

    return {"intent": "wardrobe", ...}

elif any(keyword in normalized for keyword in outfit_keywords):

    return {"intent": "outfit", ...}

```

4.2 工具定义

4.2.1 天气工具

```

@tool

def get_weather(city: str) -> Dict:
    """
    获取指定城市实时天气，返回结构化数据。

    参数:
        city: 城市名称（如"北京"、"上海"）

    返回:
        {

```

```

        "temp": 温度（摄氏度），

        "condition": 天气状况（晴/雨/多云等），

        "humidity": 相对湿度（%），

        "wind_speed": 风速描述,

        "uvi": 紫外线指数,

        "tips": 穿搭建议提示

    }

    """

```

4.2.2 衣橱工具

工具函数	功能	参数
add_clothing	添加衣物	name, category, color, material, temp_range, occasion_tags
delete_clothing	删除衣物	clothing_id
list_wardrobe	列出所有衣物	无
update_clothing	更新衣物信息	clothing_id, 可选字段
search_wardrobe	基于规则检索	temp, condition, occasion, limit

智能检索逻辑

```

@tool

def search_wardrobe(temp: int, condition: str, occasion: str = "casual", limit: int = 8) -> List[Dict]:

    """基于规则过滤检索衣橱"""

    for item in wardrobe:

        temp_match = temp_min <= temp <= temp_max

```



```

occasion_match = occasion in tags

if temp_match and occasion_match:

    primary_candidates.append(item) # 温度+场合都匹配

elif temp_match:

    secondary_candidates.append(item) # 仅温度匹配

```

4.3 配置文件

```

class Config:

    """应用配置类"""

    # LLM 配置

    DEEPSEEK_API_KEY = os.getenv("DEEPSEEK_API_KEY")

    DEEPSEEK_BASE_URL = "https://api.deepseek.com/v1"

    DEEPSEEK_MODEL = "deepseek-v4-flash"

    # 天气 API 配置

    HEWEATHER_KEY = os.getenv("HEWEATHER_KEY")

    # 缓存配置

    WEATHER_CACHE_TTL = 600 # 10 分钟缓存

    # 存储配置

    STORAGE_DIR = os.path.join(os.path.dirname(__file__), "storage")

```

```
WARDROBE_FILE = os.path.join(STORAGE_DIR, "wardrobe.json")

# LLM 参数

INTENT_CLASSIFIER_TEMPERATURE = 0.3 # 低温度保证分类准确性

FASHION_ADVISOR_TEMPERATURE = 0.5 # 中等温度增加创意性
```

五、测试与评估

5.1 功能测试

5.1.1 天气查询

询问类似“今天天气如何”等天气查询类问题，应正确调用和风天气 API 并返回当前天气信息。



今天天气如何



当前天气：阴，温度 29°C 天气温暖，短袖T恤即可

5.1.2 衣橱管理

输入“添加 衬衫，上衣，白色，棉 ”，应解析参数并写入 wardrobe.json，返回提示信息



添加 衬衫，上衣，白色，棉



已成功添加衣物：衬衫

5.2 Agent 行为评估

询问"今天穿什么", 应识别意图为 `outfit`, 场合默认为 `casual`, 进入穿搭建议链路。

 今天穿什么

 下午有面试穿什么

 好的! 以下是针对您面试场合的穿搭建议:

天气情况: 今日阴天, 29°C, 温暖但不闷热, 适合穿着轻便的正式装束。

推荐搭配方案:

- 上衣: 白衬衫 (您已有的)
- 裤子: 深蓝西裤 (您已有的)
- 鞋子: 黑皮鞋 (您已有的)

搭配理由: 白衬衫与深蓝西裤是经典的面试组合, 给人以干练、专业的印象。黑色皮鞋提升整体正式度, 与深蓝西裤形成稳重色调。29°C下, 棉质白衬衫和羊毛西裤透气性良好, 既保持体面又舒适。无需添加额外衣物, 您已有的单品即可完美应对。祝面试顺利!

询问"下午有面试穿什么", 应识别意图为 `outfit`, 提取场合为 `interview`, 触发天气查询和正式场合检索。

六、系统升级与扩展

本系统通过抽象接口预留了扩展空间。存储层当前使用 `JSON` 文件, 未来可替换为 `SQLite` 或 `PostgreSQL`, 接口已抽象, 切换成本较低。检索层目前是规则过滤, 若后续需要支持语义查询, 可接入向量数据库, 但需引入 `Embedding API`, 成本中等。近期计划先实现偏好记忆, 让用户能保存不喜欢的颜色材质并在检索时排除, 同时优化多轮对话中的指代理解问题。中期考虑接入向量检索, 支持更灵活的自然语言查询。远期若有多用户需求, 需重新设计数据隔离方案, 当前暂不投入。

七、课程总结

通过本次项目, 我理解了 `LangGraph` 状态机编排的基本用法, 实践了 `Agent` 与 `Tool` 的职责分离, 也体会到 `Prompt` 调优比写代码更耗时。规格驱动开发减少了返工, 但写规格本身也需要时间, 需权衡投入。`AI IDE` 能加速代码生成, 但 `API` 联调和 `Prompt` 迭代仍需人工把控, 不能过度依赖。记录每步耗时后发现天气 `API` 是主要瓶颈, 缓存有效。这次经历让我意识到 `Agent` 开发与

传统开发的核心差异：LLM 输出是概率性的，约束设计比功能堆叠更重要，兜底机制是常态而非例外。对课程的建议是增加 Prompt 调优和真实 API 联调的实践环节，以及补充基础的 Agent 评估方法。